Librerías JavaScript aplicadas al proyecto Tacos Joya de Cerén: interactividad y componentes dinámicos

Diego Armando Henríquez Fuentes HF261938

Josué Israel Sandoval Alfaro SA260082

Carlos Ascencio RA262936

José Ricardo Sandoval Hernández SH262272

Alexander Josué Menjívar MG202784

Nicolas Alejandro Trigueros Molina TM253158

Sistema Computacional

Universidad Don Bosco Virtual

DAW901: Desarrollo de Aplicaciones Web con Software Interpretados en el Cliente

Alexander Alberto Siguenza Campos

21 de octubre de 2026

Entrega del Foro 1

Etapa 3 Primer avance

Presentamos los enlaces del proyecto Tacos Joya de Cerén y la investigación grupal sobre las librerías JavaScript utilizadas en este avance.

Repositorio y código fuente

https://github.com/Jexxe0012/tacos-joya-de-ceren

El código integrado está en la rama main, con las pantallas HTML, los estilos CSS y la lógica JavaScript de los módulos.

Video de demostración

https://youtu.be/ShDF-nkBVLs

Aplicación web

https://jexxe0012.github.io/tacos-joya-de-ceren/pages/login.html

Investigación grupal

Se incluye a partir de la siguiente página. Explica el uso de Chart.js, SweetAlert2 y jsPDF, las validaciones, las dificultades encontradas y sus soluciones, con capturas, fragmentos de código y referencias.

Alcance del avance

Incluye gráficas y filtros de reportes, exportación a PDF, diálogos y validaciones compartidos. La aplicación distingue los roles de administrador y empleado y conserva los datos en localStorage.

Librerías JavaScript aplicadas al proyecto Tacos Joya de Cerén

Tacos Joya de Cerén es un negocio familiar de comida mexicana. En la Etapa 1 identificamos que el control de recetas, ventas, inventario y desperdicios se llevaba de forma manual, lo que dificultaba saber cuánto se gasta en cada taco y cuánto se pierde por merma. En la Etapa 2 construimos una aplicación web con HTML5, CSS3, Bootstrap 5.3 y JavaScript sin framework, que guarda los datos en el navegador con localStorage y diferencia los permisos del administrador y del empleado.

En este avance de la Etapa 3 el objetivo fue que la aplicación respondiera mejor a quien la usa y que mostrara información útil para decidir. Para eso incorporamos tres librerías externas: Chart.js para las gráficas de reportes, SweetAlert2 para los mensajes y confirmaciones, y jsPDF para descargar el reporte en PDF. También reforzamos las validaciones de los formularios combinando la validación nativa del navegador con una capa propia de reglas de negocio.

En cada sección explicamos qué es la librería, por qué la elegimos, cómo la aplicamos en el proyecto, con capturas y fragmentos del código, y qué limitaciones encontramos. Al final comparamos algunas decisiones con alternativas que también existen en el proyecto y resumimos las dificultades de cada integrante.

Chart.js

Qué es y por qué la elegimos

Chart.js es una librería de código abierto que dibuja gráficas sobre el elemento canvas de HTML5 (Chart.js, s. f.). Ofrece gráficas de líneas, barras y otros tipos, se adapta al tamaño de la pantalla y se configura con un objeto de JavaScript. Usamos la versión 4.5.1, cargada desde la red de distribución jsDelivr con su versión fija, para que una actualización de la librería no cambie el comportamiento de la aplicación sin que lo sepamos.

La elegimos porque el requerimiento 1 del foro pide un módulo de reportes con gráficas que ayuden a decidir, y Chart.js resuelve los tipos que necesitábamos sin instalar herramientas de compilación, algo importante en un proyecto de JavaScript sin framework publicado en GitHub Pages. Además permite actualizar los datos de una gráfica existente, lo que evita volver a crearla cada vez que se cambia un filtro.

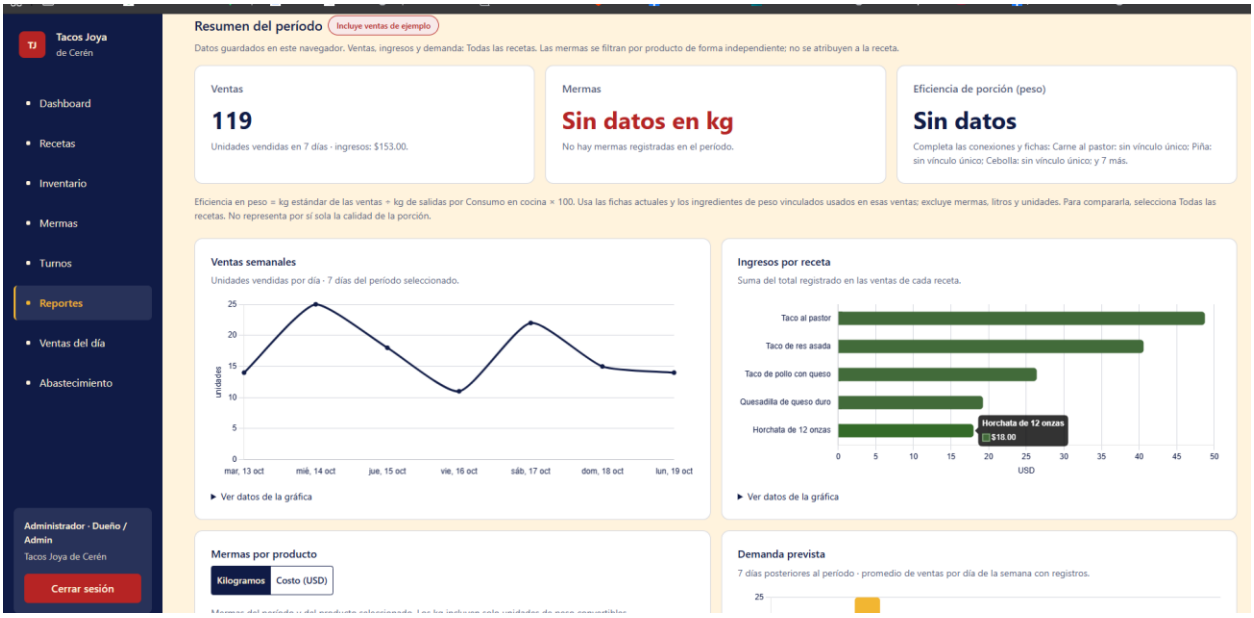
Cómo la aplicamos

En el módulo de Reportes las gráficas leen los datos que guarda el propio sistema: ventas, mermas, movimientos de inventario y recetas. La venta por día se muestra como línea; los ingresos por receta, como barras horizontales (Figura 1); las mermas por producto, en kilogramos o en dólares según el botón que se elija; y la demanda prevista de los siete días siguientes, como barras. Los colores no se escriben a mano: se toman de las variables de la paleta definidas en main.css, de modo que las gráficas mantienen la identidad visual de la aplicación.

Cuando el usuario cambia el rango de fechas, la función actualizarGraficaReporte reutiliza la gráfica que ya existe y solo reemplaza sus etiquetas y valores (Figura 2). Si en el período no hay datos, se muestra el mensaje "Sin datos en este período" en lugar de una gráfica vacía. Si la librería no carga, los mismos valores siguen disponibles en tablas, de modo que el administrador nunca se queda sin la información.

Figura 1

Módulo de Reportes con las gráficas de ventas e ingresos



Nota. Captura propia de la aplicación publicada en GitHub Pages.

Figura 2

Actualización de una gráfica existente al cambiar los filtros

```
// js/reportes-graficas.js (fragmento)
if (graficasReportes.has(tipo)) {
  const grafica = graficasReportes.get(tipo);
  grafica.data.labels = etiquetas;
  Object.assign(grafica.data.datasets[0], { data: valores, label:
unidad });
  grafica.resize();
  grafica.update("none");   // sin animación: el cambio es inmediato
  return;
}
```

Nota. Fragmento del repositorio del proyecto.

En el Dashboard se implementó una gráfica de barras con Chart.js que muestra las ventas registradas durante los últimos siete días (Figura 3). La gráfica utiliza los datos almacenados en el sistema y presenta la cantidad de tacos vendidos por día, lo que facilita comparar las ventas e

identificar los días con mayor o menor actividad. Durante las pruebas funcionales se registraron ventas desde el módulo correspondiente y se comprobó que los datos se reflejaban en la gráfica. Esta funcionalidad facilita el seguimiento de las ventas y apoya la toma de decisiones.

Figura 3

Ventas de los últimos siete días en el Dashboard



Nota. Captura propia de la aplicación publicada en GitHub Pages.

Limitaciones

Una gráfica dibujada en canvas es una imagen para el lector de pantalla, por lo que no basta para personas con discapacidad visual. Lo resolvimos acompañando cada gráfica con una tabla de valores. Otra limitación es que con 90 días de datos las etiquetas no caben; por eso el contenedor de la gráfica de ventas se desplaza horizontalmente cuando el período tiene más de siete días.

SweetAlert2

Qué es y por qué la elegimos

SweetAlert2 es una librería que reemplaza las ventanas alert, confirm y prompt del navegador por diálogos personalizables y accesibles (SweetAlert2, s. f.). Sus diálogos devuelven

una promesa, así que el código puede esperar con await la respuesta del usuario antes de continuar. Usamos la versión 11.26.25 (Monte, 2026), cargada desde jsDelivr con el atributo integrity, que hace que el navegador rechace el archivo si su contenido no coincide con el esperado (MDN Web Docs, s. f.-d).

Los diálogos nativos del navegador no se pueden adaptar a la paleta de la aplicación, bloquean toda la página y su aspecto cambia entre navegadores. El criterio 2 de la rúbrica valora que cada acción tenga una retroalimentación clara y consistente, por eso decidimos que ningún módulo llame a SweetAlert2 directamente: todos usan una capa compartida.

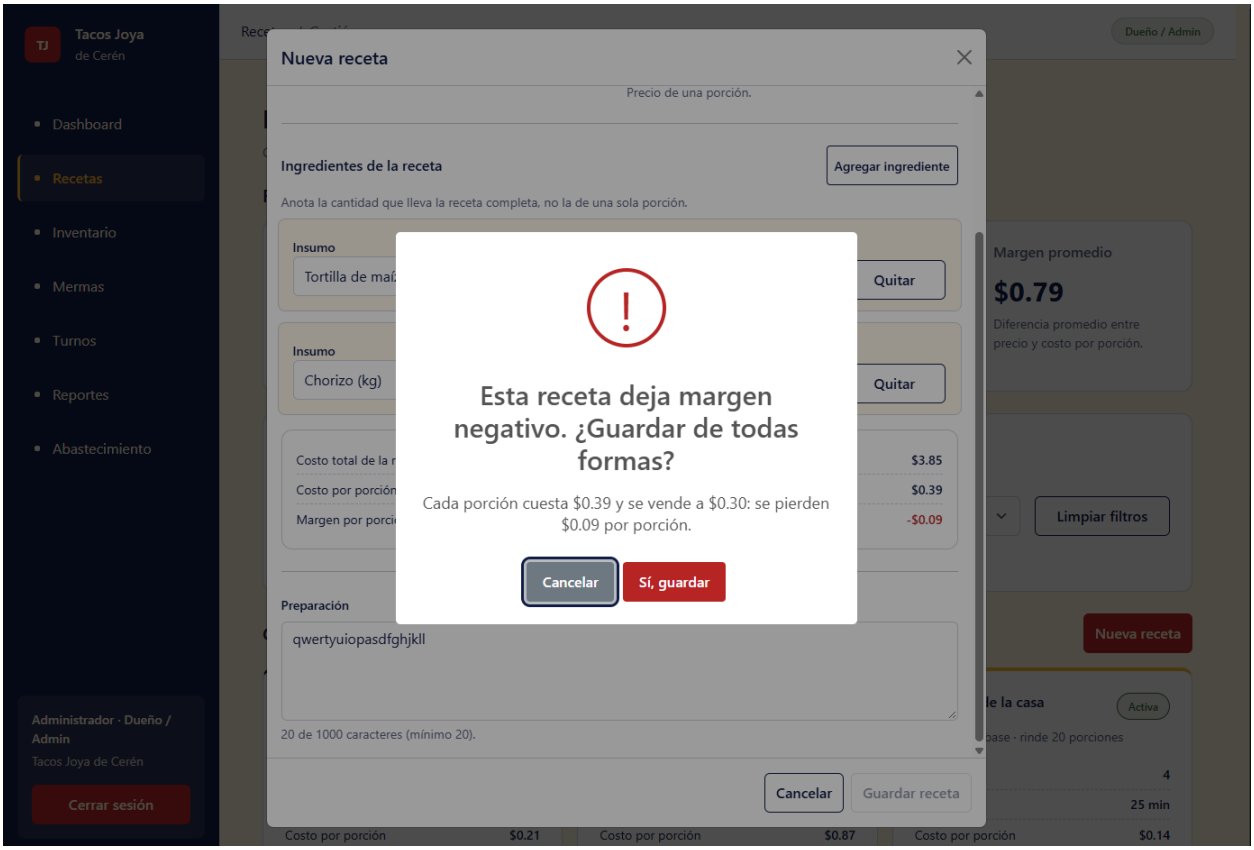
Cómo la aplicamos

La capa UI expone cuatro funciones: UI.exito para confirmar que algo se guardó, UI.error para errores que el usuario debe leer, UI.aviso para notificaciones cortas que no interrumpen y UI.confirmar para acciones importantes. Los colores salen de la paleta y los textos se pasan como texto plano (titleText), no como HTML, porque muchos mensajes incluyen datos escritos por el usuario, como el nombre de una receta, y así se evita que ese texto se interprete como código. Si el CDN falla, la capa muestra un aviso simple dentro de la página.

En el módulo de Recetas, la confirmación se usa en una regla de negocio. Si el precio de venta queda por debajo del costo por porción, antes de guardar aparece un diálogo que indica cuánto se pierde en cada porción (Figuras 4 y 5). Se permite guardar, porque puede tratarse de una promoción, pero el administrador tiene que confirmarlo. Al eliminar una receta, el diálogo advierte que los reportes dejarán de contar su consumo estándar y, si ya tiene ventas registradas, lo menciona.

Figura 4

Confirmación de margen negativo en el módulo de Recetas



Nota. Captura propia de la aplicación publicada en GitHub Pages.

Figura 5

Uso de UI.confirmar antes de guardar una receta

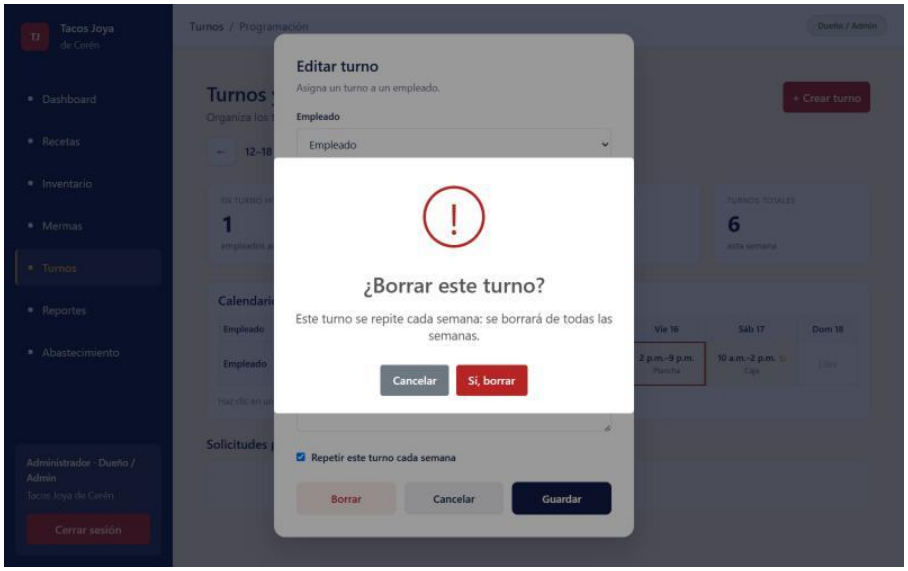
```
// js/recetas.js (fragmento)
const margen = margenPorPorcion(datos);
if (margen !== null && Math.round(margen * 100) < 0) {
  const seguir = await UI.confirmar({
    titulo: "Esta receta deja margen negativo. ¿Guardar de todas
formas?",
    texto: `Se pierden ${moneda.format(-margen)} por porción.`,
    boton: "Sí, guardar", peligro: true
  });
  if (!seguir) { campoPrecio.focus(); return; }
}
```

Nota. Fragmento del repositorio del proyecto, simplificado en el texto del mensaje.

En los demás módulos la capa se aplica igual. En Login, Turnos e Inventario se usa tanto para avisar como para confirmar. En Turnos, borrar un turno pide confirmación con peligro: true, porque cuando el turno se repite cada semana se borra de todas las semanas y el administrador debe saberlo antes de aceptar (Figura 6). En Inventario, cuando una salida deja el producto en su mínimo o por debajo, UI.aviso muestra una advertencia con la existencia que queda, para que se note en el momento y no al revisar el listado más tarde.

Figura 6

Confirmación de SweetAlert2 antes de borrar un turno semanal



Nota. Captura propia del módulo Turnos, vista de administrador. Como el turno se repite cada semana, el mensaje advierte que se borrará de todas.

En Ventas, SweetAlert2 confirma el pedido antes de guardarlo y muestra el total para evitar registros accidentales; al finalizar, confirma que la venta quedó registrada. En Abastecimiento, muestra una alerta de error cuando la cobertura calculada es crítica (menos de tres días), indicando las existencias estimadas y la fecha sugerida para hacer el pedido.

En Mermas, registrar una pérdida descuenta producto del inventario, por eso antes de guardar UI.confirmar indica cuánto se descontará y de qué producto. Si mientras el diálogo estaba abierto la existencia cambió y ya no alcanza, el registro se detiene con UI.error; al terminar, UI.exito confirma la cantidad que se descontó.

Limitaciones

SweetAlert2 muestra un solo diálogo a la vez: si se abre otro, reemplaza al anterior, por lo que dos mensajes seguidos deben esperarse con await. También encontramos un conflicto con los modales de Bootstrap, que se explica en la sección de comparación.

jsPDF

Qué es y por qué la elegimos

jsPDF es una librería que genera documentos PDF directamente en el navegador, sin un servidor (jsPDF, s. f.). Permite escribir texto, dibujar figuras, insertar imágenes y agregar páginas. Usamos la versión 4.2.1 en su versión UMD, que se carga con una etiqueta script y queda disponible en window.jspdf.

La elegimos porque la aplicación no tiene servidor: se publica como sitio estático en GitHub Pages. El administrador necesitaba poder guardar o enviar el reporte del período, y jsPDF permite armarlo en el mismo navegador con los datos que ya están cargados.

Cómo la aplicamos

El botón "Descargar reporte PDF" de Reportes crea un documento tamaño A4 con encabezado (nombre del negocio, período, fecha de generación y usuario), una tabla de indicadores, el hallazgo operativo, las tablas de resumen y las gráficas. Para las gráficas, el código crea un canvas fuera de la pantalla, dibuja la gráfica con Chart.js al doble de resolución, la convierte en imagen con toDataURL (MDN Web Docs, s. f.-a) y la inserta con addImage

(Figura 7). Cuando el texto no cabe en la página, se agrega una nueva con el mismo encabezado, y al final se numeran todas las páginas (Figura 8).

Figura 7

Inserción de una gráfica como imagen en el reporte PDF

```
// js/reportes-pdf.js (fragmento)
const grafica = imagenGrafica(parte, unidad, tipo, horizontal, limite,
                              variableColor);
// conserva la proporción de la imagen
const altura = contenido * grafica.alto / grafica.ancho;
espacio(altura + 28);   // si no cabe, pasa a una página nueva
titulo(tituloBase);
pdf.addImage(grafica.imagen, "PNG", margen, y, contenido, altura,
             undefined, "FAST");
y += altura + 6;
```

Nota. Fragmento del repositorio del proyecto.

Figura 8

Primera página del reporte PDF descargado



Tacos Joya de Cerén

Reporte de operación 13/10/2026 - 19/10/2026

Generado: 2026-10-19 22:25 (hora local)
Usuario: Administrador
Receta vendida: Todas las recetas
Producto de merma: Todos los productos
Ventas, ingresos y demanda: Todas las recetas. Las mermas se filtran por producto de forma independiente; no se atribuyen a la receta.

Resumen del período

Indicador	Valor
Unidades vendidas	119
Ingresos registrados	$153.00
Merma con peso conocido (kg)	Sin datos
Costo estimado de merma	Sin datos
Eficiencia de porción en peso	Sin datos

Eficiencia: Completa las conexiones y fichas: Carne al pastor: sin vínculo único; Piña: sin vínculo único; Cebolla: sin vínculo único; y 7 más.
El costo de merma usa los precios actuales de Inventario. Los litros, cajas y unidades sin un peso conocido no se suman en kg.
El período incluye 7 ventas marcadas como ejemplo.

Hallazgo operativo

Sin registros para analizar

No hay mermas del producto y período seleccionados.
Selecciona otro período o registra las mermas para analizarlas.

Ingresos por receta - resumen

Nota. Captura propia del archivo reporte-tjc-AAAA-MM-DD.pdf.

Mientras se genera el archivo, el botón se desactiva con Valida.bloquearEnvio para evitar descargas dobles; al terminar se muestra UI.aviso("Reporte descargado"), y si algo falla, UI.error. Si jsPDF no carga, el botón queda desactivado con un mensaje que explica por qué.

Limitaciones

Las fuentes estándar de PDF no incluyen todos los caracteres especiales, por lo que el código reemplaza los espacios especiales antes de escribir. Las gráficas quedan como imagen, así que en el PDF no se puede seleccionar su texto. Para que los períodos largos sigan siendo legibles, las gráficas se dividen en partes de 15 días o de 8 productos.

Validación de formularios

La validación nativa del navegador

HTML5 permite declarar restricciones directamente en los campos con atributos como required, min, max, minlength y maxlength, y el navegador ofrece una API para revisarlas (MDN Web Docs, s. f.-b). En el proyecto estos atributos son la primera barrera: limitan lo que se puede escribir y documentan en el HTML qué espera cada campo. Sin embargo, los mensajes del navegador no se pueden adaptar a la aplicación y no conocen las reglas del negocio; por eso cada formulario lleva el atributo novalidate y la revisión completa la hace JavaScript.

La capa compartida Valida

La capa js/validacion.js reúne reglas reutilizables (Valida.texto, Valida.numero y Valida.fecha) que devuelven un texto vacío si el valor es correcto o el mensaje de error. Para mostrar el error usa las clases is-invalid e invalid-feedback de Bootstrap (Bootstrap Team, s. f.-b), junto al campo y no en una ventana aparte. La función Valida.enVivo revisa cada campo al salir de él, quita el error en cuanto se corrige y elimina los espacios sobrantes (Figura 9). Como equipo definimos que una validación avanzada debe incluir, además de los campos obligatorios, al menos una regla que cruce datos del sistema y el bloqueo del doble envío.

Figura 9

Reglas del formulario de Recetas con Valida.enVivo

```
// js/recetas.js (fragmento)
validarCampos = Valida.enVivo(formularioReceta, {
  "receta-nombre": v => Valida.texto(v, {
    min: 3, max: 60, patron: PATRON_NOMBRE,
    mensaje: "Usa letras, números, espacios y los signos . , ' ( ) & -"
  }) || nombreRepetido(v),
  "receta-precio": reglaPrecio,
  "receta-preparacion": v => Valida.texto(v, { min: 20, max: 1000 })
});
```

Nota. Fragmento del repositorio del proyecto; se omitieron algunos campos.

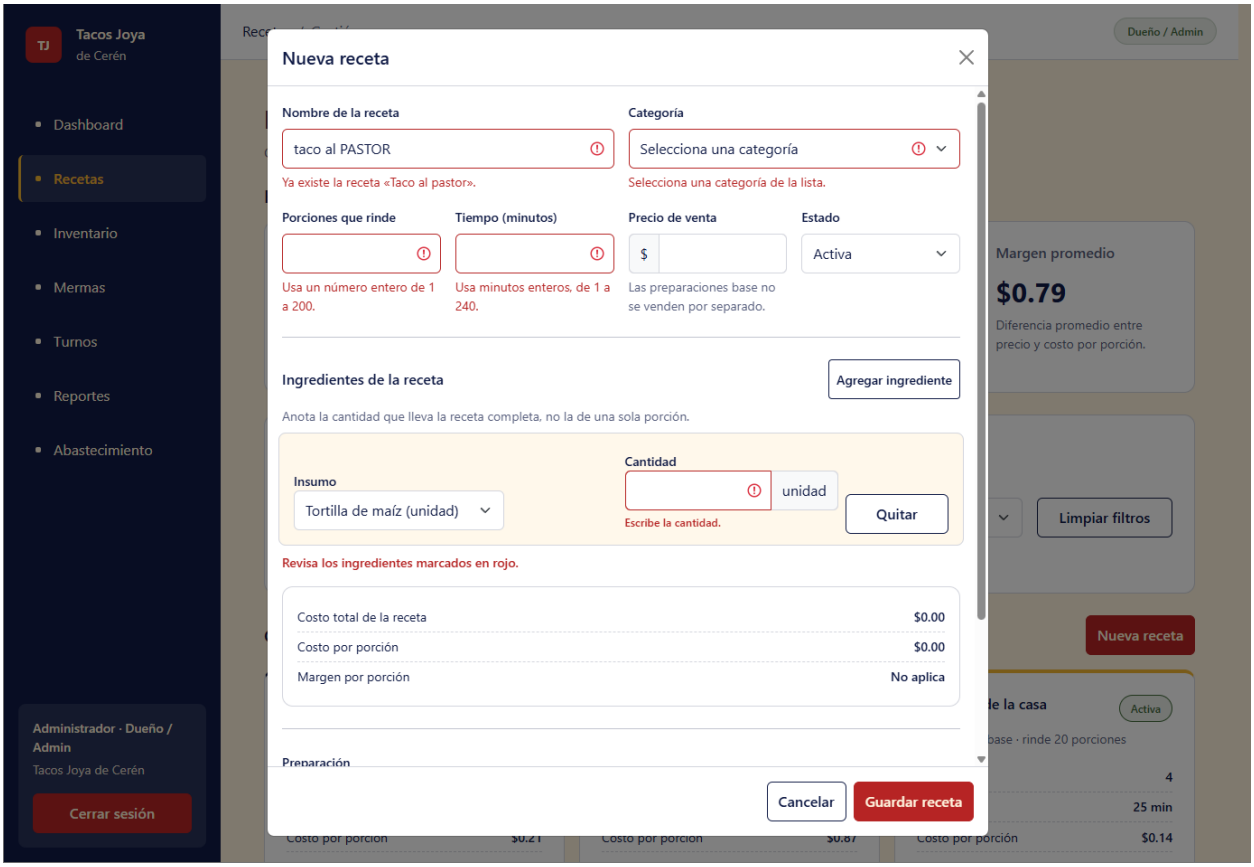
Algunas reglas que cruzan datos son: el nombre de una receta no puede repetirse, aunque cambien las mayúsculas, las tildes o los espacios, para lo cual el texto se normaliza con normalize("NFD") (MDN Web Docs, s. f.-c); un insumo que se cuenta por unidades, como la tortilla, solo acepta números enteros; y un mismo insumo no puede aparecer dos veces en la receta (Figura 10).

En Turnos, un empleado no puede tener dos turnos que se crucen el mismo día. Para comprobarlo no basta con revisar los turnos guardados de esa fecha: hay que incluir también los turnos recurrentes que caen en ella, porque un turno semanal no está guardado como un registro de ese día concreto.

En Ventas, la fecha debe ser hoy o una fecha anterior, y no puede tener más de siete días de antigüedad. El campo también limita el calendario a ese rango; la regla Valida.fecha({ noFutura: true, maxDiasAtras: 7 }) vuelve a comprobarlo al enviar el formulario y muestra el error junto al campo. Además, cada línea solo acepta recetas activas y la cantidad debe ser un número entero de 1 a 500. En Mermas, la cantidad perdida no puede superar la existencia del producto en Inventario, los productos que se cuentan por unidades solo aceptan enteros y, si el motivo es “Otro”, la observación es obligatoria. En Reportes, el rango personalizado no admite fechas futuras, la fecha final no puede ser anterior a la inicial y el período no puede pasar de 90 días.

Figura 10

Formulario de Recetas con errores marcados por campo

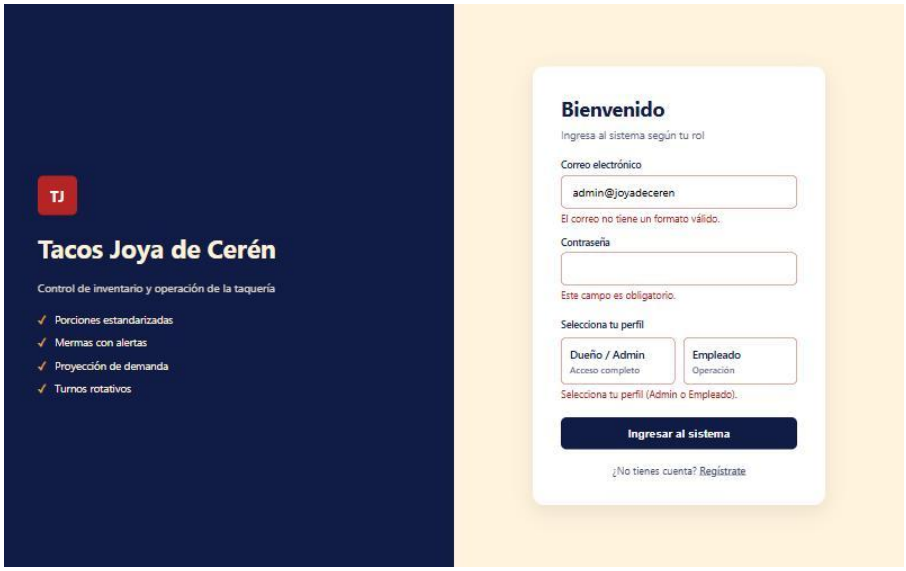


Nota. Captura propia de la aplicación publicada en GitHub Pages.

En el inicio de sesión se aplica el mismo criterio: cada error aparece debajo del campo que lo causó, sin ventanas aparte (Figura 11).

Figura 11

Errores por campo en el inicio de sesión



Nota. Captura propia. Cada error aparece debajo del campo que lo causó: formato de correo, contraseña vacía y perfil sin elegir.

Limitaciones

Toda esta validación ocurre en el navegador y los datos se guardan en localStorage, por lo que un usuario con conocimientos técnicos podría modificarlos desde las herramientas de desarrollo. En una versión con servidor, las mismas reglas tendrían que revisarse también del lado del servidor.

Comparación con alternativas presentes en el proyecto

Chart.js frente a barras hechas con CSS

En Abastecimiento, la demanda esperada por día se dibuja con barras de HTML y CSS: cada barra es un elemento cuya altura se calcula en porcentaje. Esta solución no depende de ninguna librería, carga más rápido y se adapta a la paleta sin configuración. Sin embargo, no tiene ejes, escalas ni etiquetas emergentes, y agregar otro tipo de gráfica implicaría programarlo

desde cero. En Reportes, donde se comparan varias series, unidades y períodos de hasta 90 días, Chart.js resultó más adecuado. La conclusión fue usar CSS para indicadores simples y Chart.js para el análisis.

SweetAlert2 frente a los modales de Bootstrap

Antes de este avance, Recetas e Inventario confirmaban la eliminación con un modal de Bootstrap escrito en el HTML de cada página. Funcionaba, pero cada módulo tenía su propio diseño y su propio código. Con SweetAlert2, una sola línea (await UI.confirmar(...)) produce el mismo diálogo en todo el sistema. Los modales de Bootstrap se mantuvieron para formularios largos, como el de una receta, donde sí hace falta una ventana con muchos campos (Bootstrap Team, s. f.-a).

Al combinarlos apareció un problema: el modal de Bootstrap retiene el foco del teclado dentro de sí, de modo que cuando SweetAlert2 abría la confirmación encima, el teclado dejaba de funcionar en el diálogo. Lo resolvimos creando el modal de recetas con la opción focus: false y poniendo el foco a mano en el primer campo.

Dificultades encontradas y cómo se resolvieron

La Tabla 1 resume la principal dificultad de cada integrante en este avance.

Tabla 1

Dificultades por integrante y su solución

Integrante y módulo	Dificultad	Solución
Diego — Login, Turnos, Inventario	Los avisos mostraban el nombre del producto o de la receta con la opción title de SweetAlert2, que interpreta su contenido como HTML. Un producto registrado con una etiqueta <img> y onerror habría ejecutado ese código al mostrarse el aviso.	Se cambió title por titleText en las cuatro funciones de js/ui.js, de modo que la protección queda en un solo lugar y ningún módulo tiene que recordarla. Se comprobó mostrando un aviso cuyo texto incluía una etiqueta <img>: el navegador la mostró como texto y no la ejecutó.

Integrante y módulo	Dificultad	Solución
Nicolás — Recetas	Al agregar un ingrediente desaparecía el botón "Guardar receta": el formulario quedaba entre modal-content y modal-body y rompía el desplazamiento del modal.	Se dio al formulario display: flex, dirección en columna y min-height: 0; el cuerpo se desplaza y los botones quedan visibles.
Nicolás — Recetas	Un precio como $1.15 se rechazaba por "más de dos decimales", porque 1.15 × 100 da 114.99999999999999 en JavaScript.	Se compara con un margen de error mínimo en lugar de una igualdad exacta.
Josué — Reportes	Recetas e Inventario usaban identificadores y unidades distintos para sus ingredientes. No se podía comparar el consumo esperado con las salidas de cocina sin saber qué registros correspondían al mismo producto.	Se compartió el catálogo y se guardaron vínculos en la colección vinculosInsumos, con conversiones automáticas de unidades y equivalencias para empaques. Si falta una conexión o salida, la eficiencia muestra "Sin datos" y explica la causa.
Carlos — Ventas, Abastecimiento	Mantener el registro de un pedido con varios productos y lograr que el historial y los totales del día lo muestren como una sola venta.	Se asigna un pedidoId común a todas las líneas del pedido, se guarda el subtotal de cada receta y el historial agrupa las líneas por ese identificador para mostrar un solo pedido con su total, productos, hora y método de pago.
Ricardo — Dashboard	Durante la integración de los cambios del Dashboard con la rama principal del proyecto, fue necesario comprobar que la gráfica de ventas mostrara correctamente los datos registrados y que las funcionalidades existentes se conservaran.	Se integraron los cambios mediante una solicitud de incorporación (*pull request*) en GitHub. Posteriormente, se actualizó la rama `main` y se realizaron pruebas funcionales para verificar la gráfica de ventas, las alertas operativas y la información del Dashboard. Se comprobó que el módulo funcionara correctamente después de la integración.
Alexander — Mermas	Algunos registros viejos de mermas guardaban la cantidad como texto con su unidad (por ejemplo, “5 kg”), y el indicador semanal mostraba NaN al sumarlos.	Se creó la función numeroSeguro, que toma solo el número del texto y devuelve 0 si no encuentra ninguno; así el indicador nunca muestra NaN y no se repite la unidad al mostrar la cantidad.

Conclusiones

Las tres librerías resolvieron necesidades concretas de la taquería y no se agregaron solo por cumplir el requisito. Chart.js convirtió los registros de ventas y mermas en gráficas que muestran qué se vende y dónde se pierde dinero; jsPDF permite llevar ese análisis fuera de la aplicación sin necesitar un servidor; y SweetAlert2, a través de una capa compartida, hizo que todo el sistema responda de la misma forma ante cada acción.

La decisión técnica más útil fue crear capas compartidas (UI y Valida) antes de que cada integrante trabajara su módulo. Así, los mensajes y las validaciones se comportan igual en todas las pantallas, y un cambio en la capa beneficia a todo el sistema. También aprendimos que cada librería trae límites que hay que prever: una gráfica en canvas necesita una tabla para ser accesible, y una librería que carga desde internet necesita un plan para cuando no carga.

En el siguiente avance integraremos React y una API externa, aprovechando que la lógica de negocio ya está separada de la presentación.

Referencias

Bootstrap Team. (s. f.-a). *Modal*. Bootstrap v5.3. Recuperado el 17 de octubre de 2026, de https://getbootstrap.com/docs/5.3/components/modal/

Bootstrap Team. (s. f.-b). *Validation*. Bootstrap v5.3. Recuperado el 17 de octubre de 2026, de https://getbootstrap.com/docs/5.3/forms/validation/

Chart.js. (s. f.). *Chart.js documentation*. Recuperado el 17 de octubre de 2026, de https://www.chartjs.org/docs/latest/

jsPDF. (s. f.). *jsPDF* [Repositorio de código]. GitHub. Recuperado el 17 de octubre de 2026, de https://github.com/parallax/jsPDF

MDN Web Docs. (s. f.-a). *HTMLCanvasElement: método toDataURL()*. Mozilla. Recuperado el 17 de octubre de 2026, de https://developer.mozilla.org/es/docs/Web/API/HTMLCanvasElement/toDataURL

MDN Web Docs. (s. f.-b). *Constraint validation*. Mozilla. Recuperado el 17 de octubre de 2026, de https://developer.mozilla.org/en-US/docs/Web/HTML/Constraint_validation

MDN Web Docs. (s. f.-c). *String.prototype.normalize()*. Mozilla. Recuperado el 17 de octubre de 2026, de https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/String/normalize

MDN Web Docs. (s. f.-d). *Subresource Integrity*. Mozilla. Recuperado el 16 de octubre de 2026, de https://developer.mozilla.org/en-US/docs/Web/Security/Defenses/Subresource_Integrity

Monte, L. (2026). *SweetAlert2* (Versión 11.26.25) [Software]. npm. https://www.npmjs.com/package/sweetalert2/v/11.26.25

SweetAlert2. (s. f.). *SweetAlert2: A beautiful, responsive, customizable and accessible replacement for JavaScript's popup boxes*. Recuperado el 17 de octubre de 2026, de https://sweetalert2.github.io/